

Segurança em Computação Supply Chain, SBOM, Infrastructure as Code e Policy as Code

Prof. Jean Everson Martina

Do pipeline seguro ao software confiável



- Na aula anterior vimos como o pipeline pode detectar problemas no código, nas dependências, nos secrets e na aplicação em execução.
- Agora o foco muda para aquilo que o pipeline constrói, empacota, assina, publica e implanta.
- Software moderno é composto por código, bibliotecas, containers, artefatos, configurações, infraestrutura e políticas.
- Um pipeline seguro não basta se ele entrega artefatos sem rastreabilidade ou infraestrutura sem controle.
- A pergunta da aula é: como saber o que estamos executando, como foi construído e se deveria estar rodando?

Software moderno é uma cadeia de suprimentos

- O software entregue não é formado apenas pelo código escrito pela equipe.
- Ele inclui bibliotecas, frameworks, dependências transitivas, imagens base, plugins, actions de CI/CD e serviços externos.
- Cada componente externo adiciona funcionalidade, mas também adiciona confiança implícita.
- A cadeia de suprimentos inclui desenvolvimento, build, empacotamento, publicação, deploy e atualização.
- Segurança moderna precisa controlar não só o que desenvolvemos, mas tudo aquilo que incorporamos, executamos e distribuímos.



Como ataques de supply chain acontecem



- Ataques de supply chain exploram componentes, processos ou relações de confiança usados para construir e entregar software.
- Eles podem envolver pacotes maliciosos, typosquatting, dependency confusion, maintainer takeover ou dependências abandonadas.
- Também podem atingir o processo de build, runners de CI/CD, artefatos, imagens de container ou scripts de instalação.
- O atacante tenta comprometer um ponto da cadeia para atingir muitos sistemas que confiam naquele componente.
- O risco é alto porque a atualização ou dependência comprometida pode parecer legítima para quem recebe.

Casos reais: pacotes maliciosos

- Log4Shell mostrou o impacto de uma biblioteca amplamente usada e muitas vezes invisível para as equipes.
- SolarWinds mostrou que comprometer o processo de build pode contaminar atualizações legítimas distribuídas a clientes.
- Ecossistemas como npm e PyPI sofrem com typosquatting, takeover de pacotes e publicação de dependências maliciosas.
- MOVEit reforçou o impacto de falhas em softwares amplamente adotados por organizações diferentes.
- O problema moderno não é apenas corrigir código; é descobrir onde o risco está escondido na cadeia.



SBOM: inventário do software



- SBOM significa Software Bill of Materials.
- É um inventário estruturado dos componentes que fazem parte de um software, incluindo nomes, versões, dependências e metadados relevantes.
- A principal pergunta que um SBOM ajuda a responder é: somos afetados por esta vulnerabilidade?
- Sem inventário, a resposta a incidentes depende de busca manual, planilhas incompletas e suposições.
- SBOM não torna o software seguro sozinho; ele cria visibilidade para análise, priorização e resposta.

CycloneDX e SPDX

- CycloneDX e SPDX são formatos usados para representar SBOMs de forma padronizada e processável por ferramentas.
- CycloneDX tem foco forte em segurança, dependências, serviços, vulnerabilidades e risco de supply chain.
- SPDX é um padrão aberto amplamente usado para descrever componentes, licenças, segurança e outros metadados.
- A diferença importante não é decorar campos, mas entender que SBOM precisa ser dado estruturado, não documento manual.
- Padronização permite que ferramentas diferentes gerem, troquem, validem e monitorem inventários de software.



SCA, SBOM e monitoramento contínuo



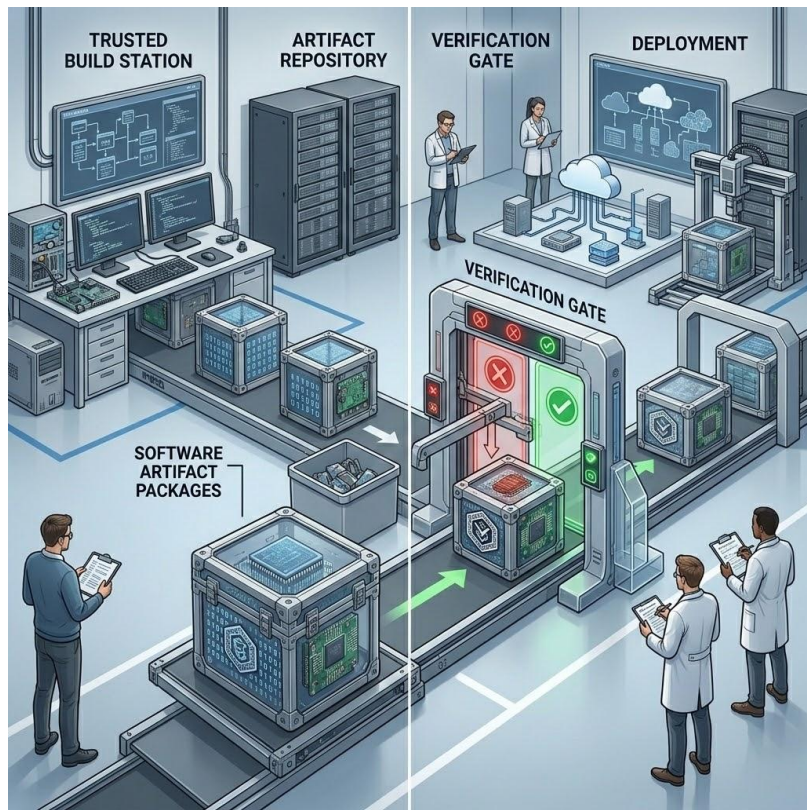
- SCA identifica componentes e vulnerabilidades conhecidas em bibliotecas, frameworks, pacotes e imagens.
- SBOM registra o inventário do que foi usado em uma versão ou artefato específico.
- Monitoramento contínuo reavalia esse inventário quando novas vulnerabilidades são publicadas.
- Um build considerado seguro hoje pode se tornar vulnerável amanhã sem que o código tenha mudado.
- Segurança de dependências é processo contínuo de descoberta, inventário, alerta, priorização e correção.

Artefatos, provenance e integridade

- Artefato é o resultado produzido pelo processo de build, como pacote, binário, imagem de container ou release.
- Provenance descreve a origem e o processo de construção de um artefato: código usado, ambiente, builder e parâmetros relevantes.
- Integridade garante que o artefato não foi alterado indevidamente entre build, armazenamento e deploy.
- SLSA organiza práticas para aumentar confiança no processo de build e reduzir adulteração na supply chain.
- Não basta saber o que foi entregue; é preciso saber como aquilo foi construído.



Assinatura de artefatos e confiança



- Assinatura de artefatos permite verificar origem e integridade de containers, pacotes, binários ou releases.
- Ferramentas como Sigstore e cosign simplificam assinatura e verificação de imagens de container e outros artefatos.
- A assinatura ajuda a responder se o artefato foi produzido por uma origem esperada e se foi alterado depois.
- Assinar não elimina vulnerabilidades no conteúdo, mas reduz risco de adulteração e distribuição não autorizada.
- Assinatura só tem valor se a verificação for obrigatória antes do deploy ou consumo do artefato.

Infrastructure as Code: infraestrutura é software

- Infrastructure as Code descreve infraestrutura por meio de arquivos versionáveis e revisáveis.
- Exemplos incluem Terraform, Ansible, Kubernetes manifests, Helm charts e Docker Compose.
- IaC permite reproduzir ambientes, revisar mudanças e automatizar provisionamento.
- Mas também permite automatizar erros de configuração em escala.
- Se configuração define comportamento do sistema, ela precisa ser tratada como parte da engenharia de software.



Falhas comuns em IaC e cloud



- Falhas de IaC e cloud muitas vezes não exploram bugs sofisticados, mas configurações inseguras.
- Exemplos comuns incluem bucket público, security group aberto, banco exposto e painel administrativo acessível.
- Containers privilegiados, imagens desatualizadas, permissões excessivas e secrets em variáveis também ampliam o risco.
- O problema pode estar em uma linha de configuração que passa pelo pipeline sem revisão adequada.
- Muitas vezes o atacante não explora uma falha complexa; ele encontra uma porta que foi publicada aberta.

Scanning de IaC e containers

- Configuração deve passar por validação automática como qualquer outro código.
- Ferramentas como Trivy, Checkov, tfsec, kube-score e kube-linter analisam arquivos de infraestrutura, containers e Kubernetes.
- Essas ferramentas ajudam a detectar permissões excessivas, imagens vulneráveis, configurações inseguras e práticas proibidas.
- O ideal é executar essas validações em pull requests, builds e antes do deploy.
- O objetivo não é gerar relatório; é impedir que configuração insegura vire infraestrutura real.



Policy as Code



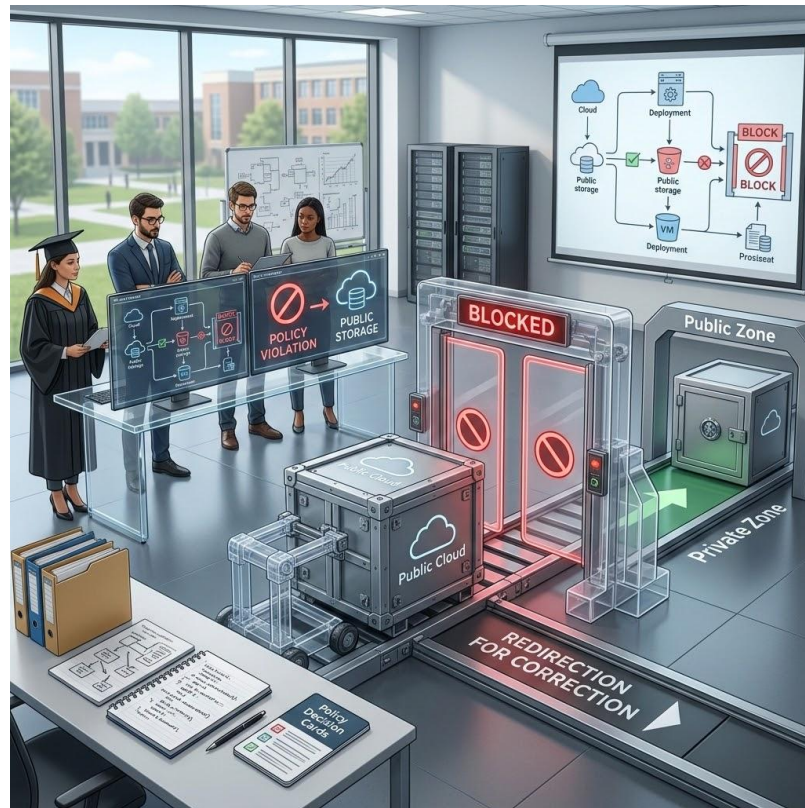
- Policy as Code transforma regras de segurança e governança em políticas executáveis, versionadas e testáveis.
- Exemplos: não permitir bucket público, bloquear container privilegiado ou impedir porta administrativa aberta para a Internet.
- Políticas passam a ser revisadas como código, aplicadas automaticamente e auditáveis no pipeline.
- OPA e Rego são referências comuns para escrever e avaliar políticas em ambientes modernos.
- A ideia central é transformar “não pode” em controle automático, consistente e verificável.

Exemplo simples de política

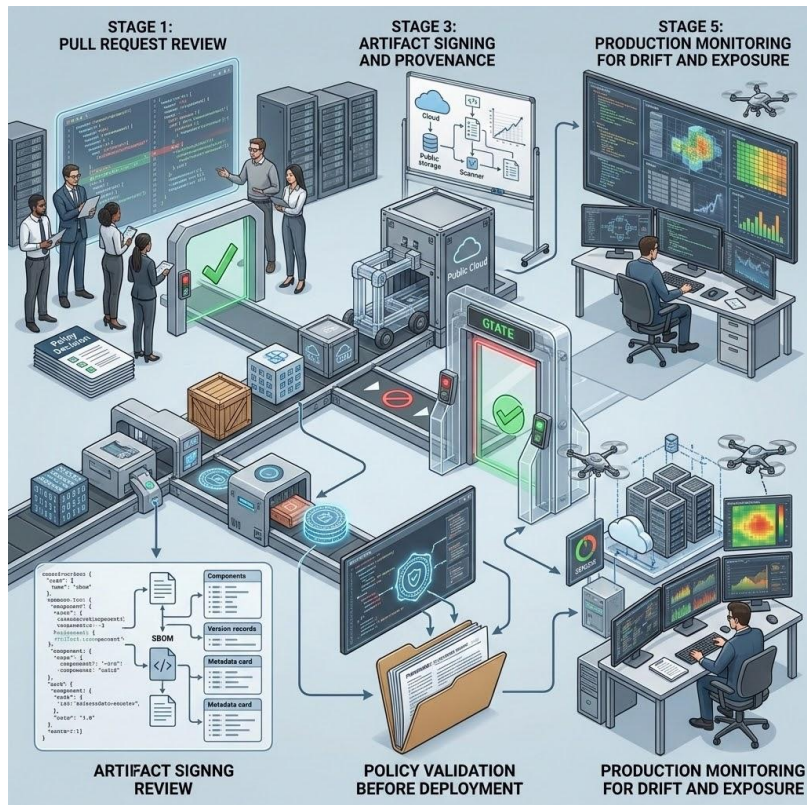
- Exemplo conceitual de política:

```
deny[msg] {  
  
    input.resource.type == "storage_bucket"  
  
    input.resource.public == true  
  
    msg := "Bucket público não permitido"  
  
}
```

- A ideia não é aprender Rego hoje, mas entender que política pode virar teste automatizado.
- Esse tipo de regra pode bloquear configuração insegura antes que ela chegue à produção.
- Policy as Code reduz dependência de revisão manual e aumenta consistência operacional.



Onde tudo entra no pipeline



- No pull request entram scans de IaC, validação de secrets, revisão de dependências e políticas básicas.
- No build entram SCA, geração de SBOM, análise de imagem, assinatura e registro de provenance.
- Antes do deploy entram verificação de assinatura, validação de políticas e aprovação conforme risco.
- Em produção entram monitoramento de drift, exposição, vulnerabilidades novas e mudanças não autorizadas.
- Aula 3 mostrou como escanear; esta aula mostra como governar o que é construído e implantado.

Fechamento: visibilidade, integridade e controle

- Supply chain ajuda a entender de onde vem o software que usamos e distribuímos.
- SBOM ajuda a saber o que existe dentro de cada versão, artefato ou produto.
- Provenance, SLSA e assinatura ajudam a verificar como o artefato foi construído e se foi alterado.
- Infrastructure as Code permite controlar infraestrutura como parte do desenvolvimento.
- Policy as Code automatiza decisões e impede que regras críticas dependam apenas de revisão manual.
- Segurança moderna depende de três perguntas: o que estou executando, como isso foi construído e quem autorizou isso a rodar?

